

Accepted 44 / 44 testcases passed

DhanaSree_2006 submitted at Aug 08, 2026 11:04

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editors — all in one plan.



Runtime

0 ms | Beats 100.00%

Memory

42.42 MB | Beats 69.10%



Code | Java

```
1 class Solution {
2     public String dayOfTheWeek(int day, int month, int year) {
3         String[] days = {
4             "Sunday", "Monday", "Tuesday", "Wednesday",
5             "Thursday", "Friday", "Saturday"
6         };
7         int[] monthDays = {
8             31, 28, 31, 30, 31, 30,
```

View more

More challenges

Code

Java Auto

```
15         if (isLeapYear(y)) {
16             totalDays++;
17         }
18     }
19     for (int m = 1; m < month; m++) {
20         totalDays += monthDays[m - 1];
21         if (m == 2 && isLeapYear(year)) {
22             totalDays++;
23         }
24     }
25     totalDays += day - 1;
26     return days[(5 + totalDays) % 7];
27 }
28
29 private boolean isLeapYear(int year) {
30     return (year % 400 == 0) ||
31            (year % 4 == 0 && year % 100 != 0);
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

day =
31

month =
8

year =
2019

Output

"Saturday"

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has 2 fields: a *name* String and a *score* integer.

Given an array of *n* Player objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, *n*, denoting the number of players.

Each of the *n* subsequent lines contains a player's *name* and *score*, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Compiler Message

Success

Input (stdin)

Expected Output

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

[Download](#)[Download](#)

Accepted 68 / 68 testcases passed

DhanaSree_2006 submitted at Aug 08, 2026 11:08

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.



Runtime

9 ms | Beats 87.14%

Memory

47.14 MB | Beats 69.29%



Code | Java

```
1 import java.util.*;
2
3 class Solution {
4     public String[] sortPeople(String[] names, int[] heights) {
5         Integer[] indices = new Integer[names.length];
6
7         // Store indices
8         for (int i = 0; i < names.length; i++) {
```

View more

More challenges

```
9         indices[i] = i;
10    }
11
12    // Sort indices based on heights in descending order
13    Arrays.sort(indices, (a, b) -> heights[b] - heights[a]);
14
15    // Create result
16    String[] result = new String[names.length];
17
18    for (int i = 0; i < names.length; i++) {
19        result[i] = names[indices[i]];
20    }
21
22    return result;
23 }
24 }
```

Saved

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1

Case 2

Input

names =
["Mary", "John", "Emma"]

heights =
[180, 165, 170]

Output

["Mary", "Emma", "John"]

Expected

["Mary", "Emma", "John"]

In computer science, a priority queue is an abstract data type which is like a regular queue, but where additionally each element has a "priority" associated with it. In a priority queue, an element with high priority is served before an element with low priority. - Wikipedia

In this problem we will test your knowledge on Java Priority Queue.

There are a number of students in a school who wait to be served. Two types of events, ENTER and SERVED, can take place which are described below.

- ENTER: A student with some priority enters the queue to be served.
- SERVED: The student with the highest priority is served (removed) from the queue.

A unique id is assigned to each student entering the queue. The queue serves the students based on the following criteria (priority criteria):

1. The student having the highest Cumulative Grade Point Average (CGPA) is served first.
2. Any students having the same CGPA will be served by name in ascending case-sensitive alphabetical order.
3. Any students having the same CGPA and name will be served in ascending order of the id.

Create the following two classes:

- The Student class should implement:
 - The constructor `Student(int id, String name, double cgpa).`
 - The method `int getID()` to return the id of the student.
 - The method `String getName()` to return the name of the student.
 - The method `double getCGPA()` to return the CGPA of the student.

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Input (stdin)

Download

```
1 12
2 ENTER John 3.75 50
3 ENTER Mark 3.8 24
4 ENTER Shafaet 3.7 35
5 SERVED
6 SERVED
7 ENTER Samiha 3.85 36
8 SERVED
9 ENTER Ashley 3.9 42
10 ENTER Maria 3.6 46
11 ENTER Anik 3.95 49
```

Problem

Sometimes it's better to use dynamic size arrays. Java's [ArrayList](#) can provide you this feature. Try to solve this problem using ArrayList.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of x^{th} line.

Take your input from System.in.

Input Format

The first line has an integer n . In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y .

Constraints

- $1 \leq n \leq 20000$
- $0 \leq d \leq 50000$
- $1 \leq q \leq 1000$
- $1 \leq x \leq n$

Each number will fit in signed integer.

Total number of integers in n lines will not cross 10^5 .

Output Format

In each line, output the number located in y^{th} position of x^{th} line. If there is no such position, just print "ERROR!"

Sample Input

Submissions

Leaderboard

Discussions

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Compiler Message

Success

Input (stdin)

Download

```
1 5
2 5 41 77 74 22 44
3 1 12
4 4 37 34 36 52
5 0
6 3 20 22 33
7 5
8 1 3
```

Accepted 236 / 236 testcases passed

DhanaSree_2006 submitted at Aug 08, 2026 11:06

Analysis

Solution

₹583.25/mo
Save over 80%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.



Runtime

6 ms | Beats 94.60%

Memory

45.06 MB | Beats 45.84%



Code | Java

```
1 import java.util.*;
2
3 class Solution {
4     public String largestNumber(int[] nums) {
5         String[] arr = new String[nums.length];
6
7         // Convert integers to strings
8         for (int i = 0; i < nums.length; i++) {
```

View more

More challenges



Java Auto

```
14
15     // If the largest number is 0, return "0"
16     if (arr[0].equals("0")) {
17         return "0";
18     }
19
20     // Build the result
21     StringBuilder result = new StringBuilder();
22
23     for (String s : arr) {
24         result.append(s);
25     }
26
27     return result.toString();
28 }
29 }
```

Saved

Testcase Test Result

Accepted Runtime: 1 ms

Case 1

Case 2

Input

```
nums =
[10,2]
```

Output

"210"

Expected

"210"

Contribute a testcase

Problem

Submissions

Leaderboard

Discussions

The `Calendar` class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method, `getDay`, which returns the day on that date. To simplify your task, we have provided a portion of the code in the editor.

Example

`month = 8`
`day = 14`
`year = 2017`

The method should return `MONDAY` as the day on that date.

AUGUST 2017						
SUN	MON	TUE	WED	THU	FRI	SAT
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19

Congratulations

You solved this challenge. Would you like to challenge your friends?

[f](#)[t](#)[in](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Compiler Message

Success

Input (stdin)

08 05 2015

Expected Output

WEDNESDAY

Download

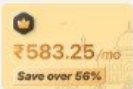
Download

Accepted 105 / 105 testcases passed

DhanaSree_2006 submitted at Aug 04, 2026 11:43

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

9 ms Beats 13.89%

Memory

43.49 MB Beats 29.17%



Code Java

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5     public int daysBetweenDates(String date1, String date2) {
6         LocalDate d1 = LocalDate.parse(date1);
7         LocalDate d2 = LocalDate.parse(date2);
8     }
}
```

View more

More challenges

Code

Java Auto

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5     public int daysBetweenDates(String date1, String date2) {
6         LocalDate d1 = LocalDate.parse(date1);
7         LocalDate d2 = LocalDate.parse(date2);
8
9         return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
10    }
11 }
12
13
14
15
```

Saved

Testcase Test Result

Accepted Runtime: 5 ms

Case 1

Case 2

Input

date1 =
"2019-06-29"

date2 =
"2019-06-30"

Output

1

Expected

1

Accepted 105 / 105 testcases passed

DhanaSree_2006 submitted at Aug 04, 2026 11:43

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.



Runtime

9 ms | Beats 13.89%

Memory

43.49 MB | Beats 29.17%



Code | Java

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5     public int daysBetweenDates(String date1, String date2) {
6         LocalDate d1 = LocalDate.parse(date1);
7         LocalDate d2 = LocalDate.parse(date2);
8     }
9 }
```

View more

More challenges

Code

Java Auto

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5     public int daysBetweenDates(String date1, String date2) {
6         LocalDate d1 = LocalDate.parse(date1);
7         LocalDate d2 = LocalDate.parse(date2);
8
9         return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
10    }
11 }
12
13
14
15
```

Saved

Testcase Test Result

Accepted Runtime: 5 ms

Case 1

Case 2

Input

date1 =
"2019-06-29"

date2 =
"2019-06-30"

Output

1

Expected

1